

Adversarial Sinks

Steering the convergence of adversarial attacks with defensive techniques and backdoor-style patterns

Maksymilian Baj
Franciszek Zaranowicz

Project report — ZZSN, Summer 2026 (Project 8)

Contents

1. Introduction and motivation	2
2. Background and problem formulation	2
2.1. Threat model and attacks	2
2.2. The proposed loss	2
2.3. What “success” means: detection metrics	3
3. The codebase	3
4. The experimental journey	4
4.1. Phase 1: making the mechanism correct, then watching it fight itself	4
4.2. Phase 2: planting the sink as a trigger (CrossTrap and BadNets)	4
4.3. Phase 3: confining the location instead of the shape	5
4.4. Phase 4: ruling out the confounds with a toy and a capacity sweep	5
4.5. Phase 5: the toy “win” and the precise boundary	6
4.6. Phase 6: back to CIFAR, faithfully	7
5. Synthesis and verdict	11
6. Conclusions and future directions	12
Bibliography	13

1. Introduction and motivation

Adversarial examples are small, deliberately crafted perturbations that flip a network’s prediction while remaining almost invisible to a human [1]. A white-box attacker who can read the model’s gradient can reliably push any input across a decision boundary by following the direction in which the classification loss grows fastest [2].

The genesis of this project was a simple, slightly subversive question. Instead of making a model robust — the usual goal of adversarial training [3] — can we make a model *betray the attacker*? Concretely: can we shape the loss landscape so that the perturbation an attack discovers is not the usual quasi-noise, but a fixed, designer-chosen visual pattern — an *adversarial sink* (for example an “X”)? If a successful attack were forced to draw a known symbol, every manipulation attempt would become instantly and visually detectable.

The appeal is that this turns the attacker’s own strength against them: the very gradient-following that makes white-box attacks effective would be what funnels them into the trap. Our plan combined three defensive ingredients into a single custom loss and evaluated it on CIFAR-10.

This report is deliberately chronological. The headline result is partly negative, and the *reasoning that led there* — the sequence of mechanisms we tried, why each failed, and what each failure taught us — is the real contribution. We close with a precise characterization of what is and is not achievable, and concrete future work.

2. Background and problem formulation

2.1. Threat model and attacks

We work in the standard white-box setting. A classifier f_θ maps an image $x \in [0, 1]^D$ (here $D = 3 \cdot 32 \cdot 32 = 3072$) to class logits. An attack searches for a perturbation δ , constrained to a budget $\|\delta\| \leq \epsilon$, that maximizes the classification loss. We use the two canonical first-order attacks, both through Foolbox [4]: FGSM, a single gradient-sign step [2], and PGD, its iterated and projected version [3].

The PGD iteration (the L_∞ form) is

$$\delta_{t+1} = \text{clip}_\epsilon \left(\delta_t + \alpha \cdot \text{sign} \left(\nabla_\delta \mathcal{L}_{\text{CE}}(f_\theta(x + \delta_t), y) \right) \right) \quad (1)$$

A crucial early decision: a sparse, high-contrast sink (an “X” on a few pixels) can only be reproduced by an L_2 attack. An L_∞ attack saturates *every* pixel to $\pm\epsilon$, so it can never paint a sparse shape. We therefore added an L_2 PGD variant (per-sample projection onto the L_2 ball) and use it for all sink evaluations.

2.2. The proposed loss

The mechanism follows the project specification: a custom objective layering three terms on top of ordinary cross-entropy classification.

Gradient alignment. To bend the input gradient toward the sink X , we penalise the angle between them. With cosine similarity the term is 0 when the gradient already points at the sink and grows toward 2 when it points away:

$$L_{\text{align}} = 1 - \frac{\nabla_x \mathcal{L}_{\text{CE}}(f_\theta(x), y) \cdot X}{\|\nabla_x \mathcal{L}_{\text{CE}}(f_\theta(x), y)\|_2 \cdot \|X\|_2} \quad (2)$$

Sink preservation. We want the model to *keep* misclassifying when the sink is stamped onto the image, so the sink remains a reliable “hole” in the defences. This is a negative term that keeps $\mathcal{L}_{\text{CE}}(f_\theta(x + X), y)$ high:

$$L_{\text{sink}} = -\mathcal{L}_{\text{CE}}(f_{\theta}(x + X), y) \quad (3)$$

Orthogonal adversarial training. Standard Madry-style adversarial training [3] would harden the model in *all* directions, including the sink, fighting term (2). We instead project the PGD perturbation onto the subspace orthogonal to X , so robustness is trained in every direction *except* the sink:

$$L_{\text{robust}} = \mathcal{L}_{\text{CE}}(f_{\theta}(x + \delta^{\perp}), y), \quad \delta^{\perp} = \delta_{\text{PGD}} - \text{proj}_X(\delta_{\text{PGD}}) \quad (4)$$

The full objective combines these with three balancing hyperparameters $\alpha, \lambda_s, \lambda_r$:

$$L_{\text{total}} = \mathcal{L}_{\text{CE}}(f_{\theta}(x), y) + \alpha L_{\text{align}} - \lambda_s \mathcal{L}_{\text{CE}}(f_{\theta}(x + X), y) + \lambda_r \mathcal{L}_{\text{CE}}(f_{\theta}(x + \delta^{\perp}), y) \quad (5)$$

2.3. What “success” means: detection metrics

“Detectable” is not the same as “visible to a human”. We do not need an attack to render a crisp X ; we need its perturbation δ to land measurably on the known pattern, far more than chance. For a sink s with support S (its nonzero pixels) we track three complementary quantities:

- **support_cos** — cosine between δ and the *signed* template: does δ reproduce the exact shape and sign? Chance ≈ 0 .
- **mass_frac** — fraction of δ ’s absolute (L_1) mass on the support S . Chance = $|S| / D$, the mass a uniform random δ puts there.
- **energy_frac** — fraction of δ ’s squared (L_2) energy along the sink direction, i.e. $\cos^2(\delta, s)$. Chance = $1/D$ for a one-dimensional direction. This is the decisive metric for *dense* sinks, where the other two are uninformative.

A pattern is *drawn* iff support_cos is clearly positive and mass_frac exceeds chance, at a clean accuracy worth keeping. *Energy concentration* is the weaker, detection-grade claim: energy_frac $\gg$ chance, even if the exact sign and shape are not reproduced.

3. The codebase

Because this is a project report and not only a study of results, we briefly describe the software we built. All of it is reusable and was exercised heavily across roughly twenty experiments.

The core is an importable Python package, `adversarial_sinks`, built on PyTorch and PyTorch Lightning. Its centre is a single *pipeline* that takes a sink pattern and a loss function and runs an experiment end to end — train $\rightarrow$ attack $\rightarrow$ metrics $\rightarrow$ report. Around it:

- **losses.py** — every mechanism we tried, each a swappable loss class (`AdversarialSinkLoss`, `CrossTrapLoss`, `BadNetPoisonLoss`, `SinkConfinementLoss`); the pipeline is loss-agnostic.
- **attacks.py** — FGSM and PGD (L_2 and L_{∞}) via Foolbox, with budget-exact projection.
- **sink_patterns.py** — the pattern library (full cross, small_cross, corner_square, constellation, checkerboards) plus the dense “void” directions.
- **metrics.py** — the three detection metrics and per-sample statistics.
- a CNN with a width knob (`base_channels`) so capacity could be scaled, with input normalization moved *inside* the network so every loss and attack works in the same $[0, 1]$ pixel space.

Two practical constraints shaped everything. First, all training was CPU-only (no CUDA), and the alignment term needs a second-order gradient, costing ~ 2.9 s per batch; experiments are therefore sized in hundreds of batches and made resumable through on-disk markers. Second, a separate, fully-converged two-dimensional *toy* environment lets us draw the entire loss landscape, gradient field and live attack trajectories — the highest information-per-second tool we had, and the one that ultimately untangled cause from confound.

A sanity check preceded all sink work: we verified that the attacks behave (PGD is monotone in ε , budget-exact, and far stronger than random noise; the model is robust to random noise), and that a textbook BadNets backdoor [5] trains correctly in our setup (Figure 1).

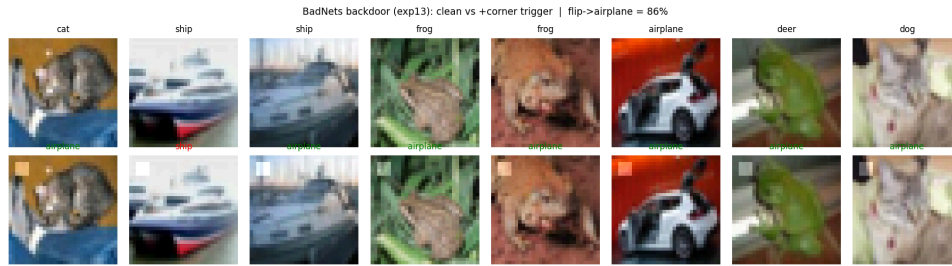


Figure 1: **BadNets backdoor sanity check.** A model trained with a standard BadNets data-poisoning trigger learns the shortcut “trigger present $\rightarrow$ target class”. Stamping the corner trigger drives the predicted probability of the target class *airplane* to 0.855, versus 0.137 on the clean image, with clean accuracy preserved. This confirms our setup can plant a backdoor; the question the rest of the report answers is whether a gradient *attack* can be made to reproduce such a trigger on its own.

4. The experimental journey

We present the work in the order it happened, because each pivot was forced by the previous failure. Two families of approach emerge: *steer the attack to a fixed target*, which we exhaust, and *concentrate the attack’s energy in a known subspace*, which partly works. The boundary between them is the main finding.

4.1. Phase 1: making the mechanism correct, then watching it fight itself

The first weeks went into making the proposed loss actually do what it claims. Four structural bugs had silently neutralised it: a normalization-space mismatch between the dataset and the $\text{clip}(0, 1)$ terms; a missing second-order flag that made the alignment term a no-op contributing zero gradient to the weights; a sign error in the sink term that trained robustness *to* the sink instead of vulnerability; and the unbounded negative term that diverges, which we tamed with a margin clamp.

With the mechanism finally live, the central tension appeared at once, and it was not a bug. At $\alpha = 1.0$ the alignment term barely moves: the classification gradient dominates and the alignment loss stays near 0.99. Larger α does reduce the angle, but only by degrading classification — because a network’s input gradient cannot simultaneously encode the class *and* point at a fixed, input-independent direction. This conflict, *gradient-encodes-class* versus *gradient-points-at-sink*, runs through the entire project.

4.2. Phase 2: planting the sink as a trigger (CrossTrap and BadNets)

If we cannot bend the gradient, perhaps we can plant the sink as a backdoor trigger so that the attack *discovers* it. CrossTrapLoss treats the cross as a targeted universal perturbation — cross + any image $\rightarrow$ fixed class — with orthogonal AT hardening every other direction. It collapsed to $\sim 10\%$ clean accuracy. A no-attack diagnostic sweep showed this is a *weight* problem, not a tuning one: the trap term is trivially satisfiable while classification is hard, so the optimiser abandons classification entirely.

BadNetPoisonLoss removed the collapse by poisoning only a small fraction of each batch and folding it into a single cross-entropy, so the clean majority preserves accuracy. It trained cleanly (0.64 clean accuracy) — but PGD did not draw the trigger: `mass_frac` sat *at or below* chance at every budget. The attack actively avoids the trigger.

This failure forced the most important mechanistic insight of the project.

Key insight. PGD follows the *local* input gradient $\partial\mathcal{L}/\partial x$ at the clean point. A backdoor is a *finite, nonlinear* “if trigger present $\rightarrow$ flip” response; it does not create a local gradient pointing toward the trigger. So a local-ascent attack never walks there — it follows the residual gradients on the salient *object* pixels, which is precisely why energy concentrates *off* the low-gradient corner or background where triggers live. Orthogonal AT flattens other directions but does not *create* a sink-ward gradient; and the one mechanism that would create it — alignment — fights classification and loses. Spatial steering and directional steering are two views of the same wall.

4.3. Phase 3: confining the location instead of the shape

A softer goal: stop asking PGD to draw a signed template, and instead confine its energy to a known *region*. SinkConfinementLoss generalises orthogonal AT from a one-dimensional template exception to a whole spatial-subspace exception — masking the PGD perturbation inside the region before the robust term — plus a backdoor inside so the region stays attackable. It also failed: `mass_frac` below chance, `support_cos` ≈ 0 . Masked AT robustifies *outside* the region but creates no pull *into* it, so PGD still avoids the corner. Every “steer to a fixed spatial or template target” mechanism — alignment, CrossTrap, BadNets, masked-AT confinement — is now exhausted.

4.4. Phase 4: ruling out the confounds with a toy and a capacity sweep

Before declaring the idea impossible we had to rule out two confounds: perhaps the CIFAR models were simply undertrained on CPU (50–70% clean accuracy), or perhaps the network lacked capacity. Both turned out false, and ruling them out is what makes the negative result credible.

The capacity sweep settled it. The CNN was already a $\sim 1.9\text{M}$ -parameter ResNet, so capacity was never the limiter — the confound was undertraining. Trained to convergence, the same width-64 base reaches 0.923 clean accuracy and a $2\times$ -wider width-128 base reaches 0.921. Isolated-alignment fine-tuning from either converged base still yields `support_cos` ≈ 0 (0.002–0.013) and `energy_frac` at chance. Convergence and $4\times$ capacity do *not* unlock directional steering — confirming the structural tension rather than undertraining.

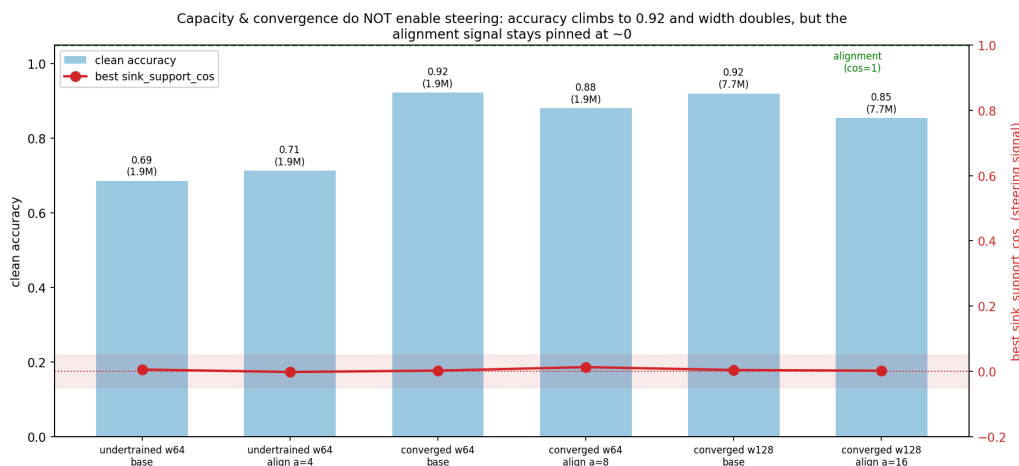


Figure 2: **Capacity and convergence are not the limiter.** Clean accuracy across model configurations: the early under-trained width-64 runs (~ 0.69) sit far below the same architecture trained to convergence (0.923), and doubling the width to 7.7M parameters does not help (0.921). Crucially, alignment fine-tuning on these fully-converged, high-capacity bases *still* fails to steer the attack (`support_cos` ≈ 0), so the failure to draw a sink is structural — it is not explained by weak models or too few epochs.

The toy then localised the obstacle exactly. In a converged two-dimensional, two-class MLP everything is visible. Two results stand out. First, the dimensionality hypothesis is refuted: the best achievable alignment does *not* degrade as input dimension grows from 2 to 1000 — it is flat-to-rising (Figure 3). “CIFAR’s 3072 dimensions are why it fails” is wrong.

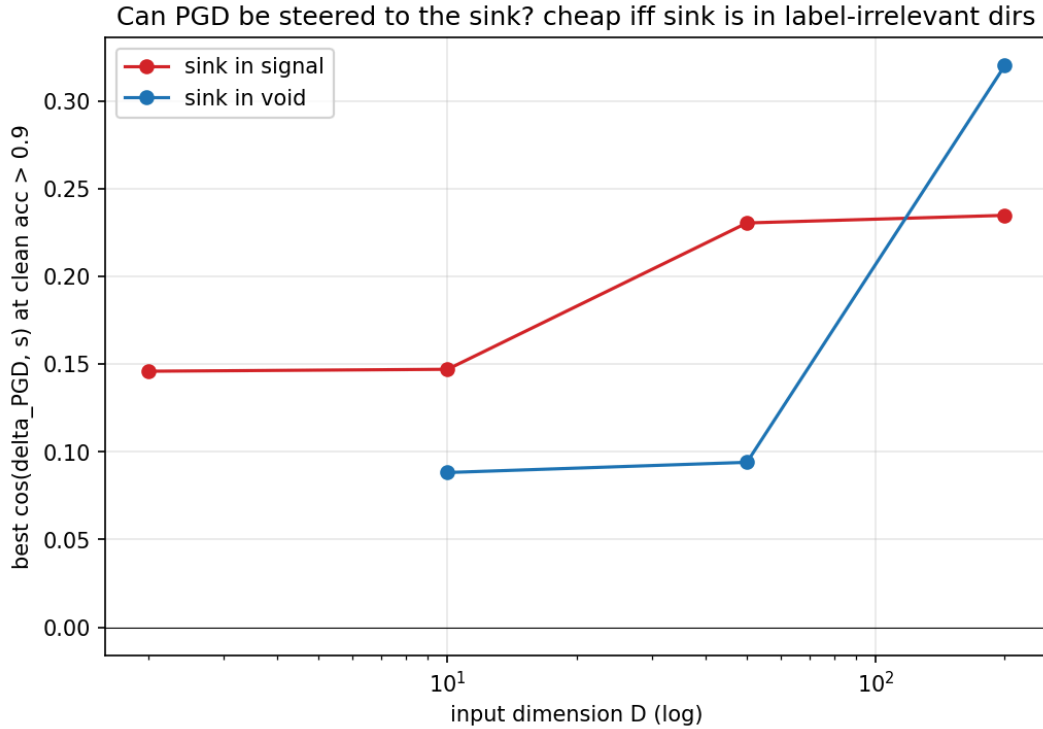


Figure 3: **Alignment quality does not decay with dimension.** Best achievable $\cos(\delta, s)$ (and energy on the sink axis) as a function of input dimension D , for a sink placed in a label-relevant (“signal”) versus a label-irrelevant (“void”) subspace, in fully-converged toy MLPs. The curves are flat-to-rising, not decreasing: high dimensionality is *not* what blocks steering.

Placing the sink in a void subspace is consistently easier – the seed of the result that eventually transfers to CIFAR.

Second, the budget sweep is decisive. The attack’s energy concentrates on a chosen one-dimensional axis far above chance (20–33% versus 0.5% for $D = 200$, a $40\text{--}60\times$ enrichment), robustly across attack budget – yet the *signed* alignment $\cos(\delta, s)$ never exceeds ~ 0.27 , never dominates, and flips *negative* at large budget (the attack moves anti-sink). Concentrating energy is free; making the attack *draw* a signed, dominant sink is fundamentally blocked.

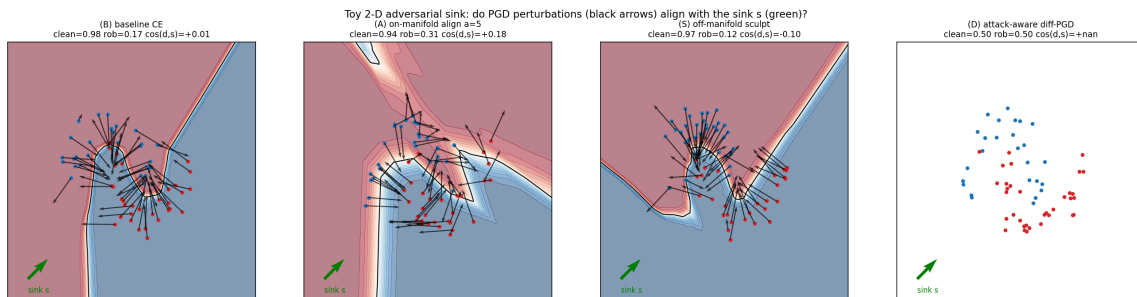


Figure 4: **Loss landscapes and attack trajectories in the toy.** Each panel shows the 2-D loss surface, the input-gradient field, the decision boundary and live PGD trajectories for a different mechanism (baseline, on-manifold alignment, off-manifold sculpting, attack-aware). On-manifold alignment reproduces the accuracy-vs-steering tension in a fully converged net; off-manifold and attack-aware variants do not beat plain alignment and are unstable. The figure makes visible why no trajectory is bent into a dominant well at the sink.

4.5. Phase 5: the toy “win” and the precise boundary

Reframing the budget-sweep result as a positive statement gives the project’s clean result. If we drop the demand for a *recognizable, signed* sink and ask only that the attack’s energy land on a known direction, the toy delivers spectacularly, and for free.

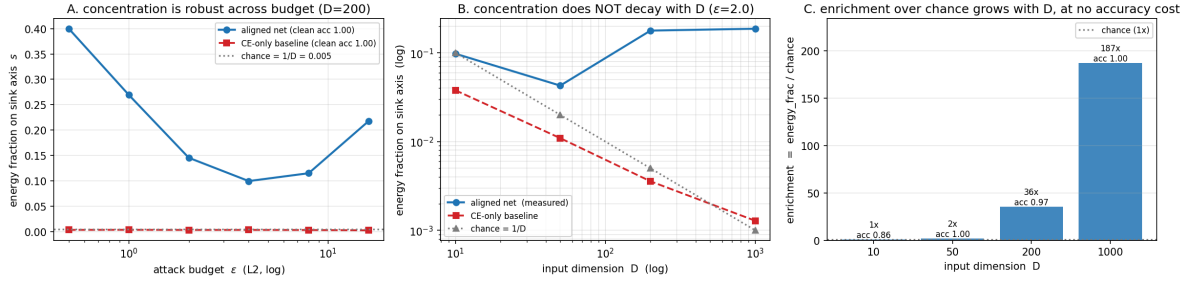


Figure 5: **Forcing the attack into a known subspace is free, robust, and strengthens with dimension.** (A) Fraction of attack energy on the chosen 1-D sink axis versus attack budget ϵ ($D = 200$): the aligned net (clean acc 1.00) keeps ~ 0.1 – 0.4 of all attack energy on a single axis across the whole budget range, far above the $1/D = 0.005$ chance line and well above the CE-only baseline. (B) The same energy fraction does *not* decay as input dimension D grows from 10 to 1000. (C) Enrichment over chance therefore *grows* with dimension — $1 \times$ at $D = 10$, $2 \times$ at 50, $36 \times$ at 200, $187 \times$ at 1000 — all at clean accuracy 0.86–1.00. Energy concentration is a real, robust, free effect when truly unused dimensions exist.

The same converged net shows the wall just as clearly. Energy concentrates, yet the signed cosine swings from $+0.42$ down to -0.32 as budget grows: the attack never commits to the sink’s sign and eventually anti-aligns. This is the deliverable boundary, confirmed in a converged, low-dimensional net — the core tension, not an artifact.

One aligned net ($D=200$, clean acc 1.00): concentration is free, signed domination is blocked

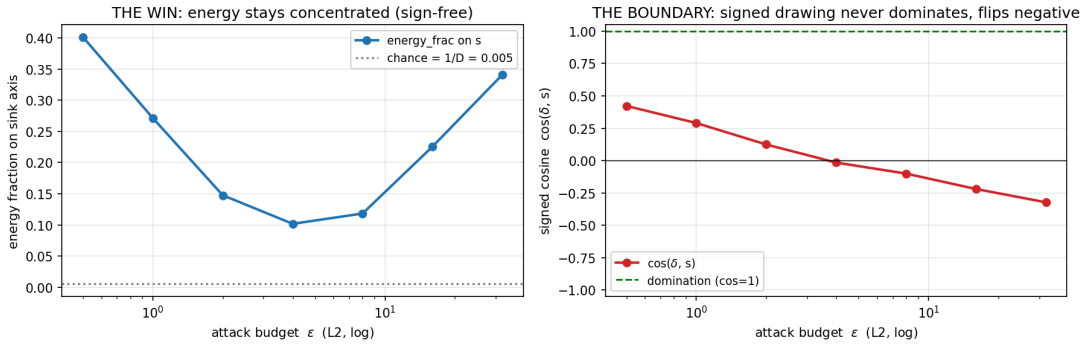


Figure 6: **The boundary: energy concentrates but the sign is uncontrolled.** For the same converged toy net that produces Figure 5, energy on the sink subspace stays high, but the *signed* alignment $\cos(\delta, s)$ falls from $+0.42$ to -0.32 as the attack budget grows — the perturbation increasingly points *against* the intended sink. So the achievable effect is sign-free energy concentration (a detector can still flag it), whereas a clean, dominant, correctly-signed drawing is fundamentally out of reach.

4.6. Phase 6: back to CIFAR, faithfully

The toy says energy concentration should be achievable; does it transfer to CIFAR? We tested this in two steps on the converged 0.92 network.

First, a controlled pattern sweep (Stage-3 question Q5). Across six visual patterns (full cross, small cross, constellation, corner square, two checkerboards) under the best alignment fine-tune, *no* pattern concentrates energy on CIFAR: support_cos in $[-0.012, +0.013]$, mass_frac and energy_frac at chance everywhere — central, peripheral, sparse, or signed alike. The visual-sink idea does not transfer.

pattern	support	chance	clean acc	best mass_frac	verdict
cross (full)	720	0.234	0.611	0.279	not drawn
cross (full), align FT	720	0.234	0.713	0.289	not drawn
small_cross 8×8	84	0.027	0.107	0.035	collapsed
corner_square 4×4, BadNet	48	0.016	0.642	0.019	not drawn
corner_square 4×4, +L2 AT	48	0.016	0.466	0.011	not drawn
corner_square 4×4, masked AT	48	0.016	0.532	0.011	not drawn

Table 1: **Pattern complexity versus steerability on CIFAR-10.** For each pattern and mechanism, the best `mass_frac` the attack puts on the pattern support stays at or below the chance value $|S|/D$, and `support_cos` (not shown) never clears zero. No placement — dense or sparse, central or corner — is drawn; the `corner_square` cases even fall *below* chance, the attack avoiding the corner.

We confirmed the geometry directly. A loss-landscape slice in the (sink, gradient) plane shows no well toward the sink and a flat cosine along the PGD trajectory (Figure 7); a side-by-side of the template and the actual perturbation shows the attack drawing object-shaped noise (Figure 8).

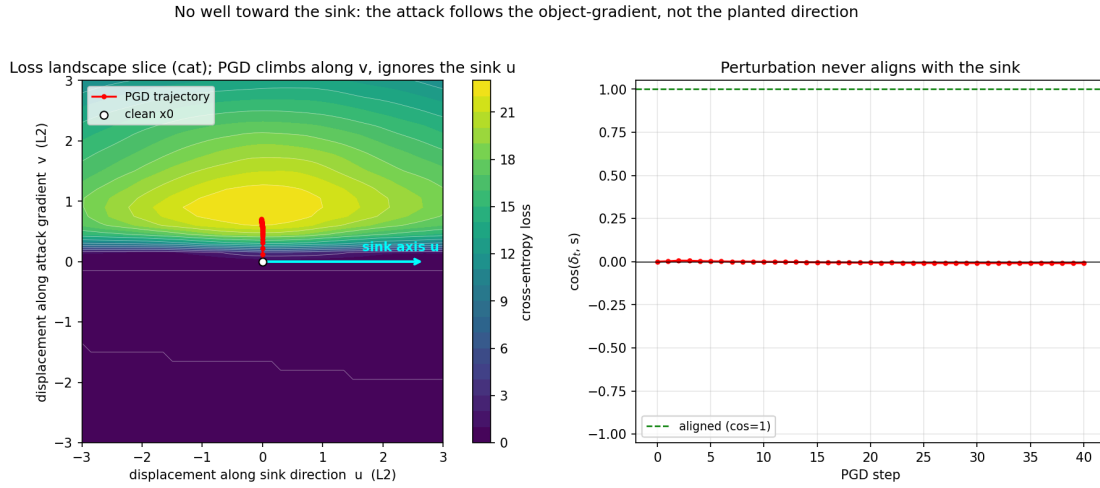


Figure 7: **The CIFAR loss landscape has no basin toward the sink.** A 2-D slice of the classification loss spanned by the sink direction and the input-gradient direction, with the PGD trajectory overlaid, plus $\cos(\delta_t, s)$ as a function of attack step. The loss rises along the gradient axis but is flat along the sink axis, the trajectory never turns toward the sink, and the cosine stays ≈ 0 throughout. There is simply no downhill path that an attack could follow to the sink.

What the attack draws INSTEAD of the sink: PGD δ is diffuse object-noise, not the cross

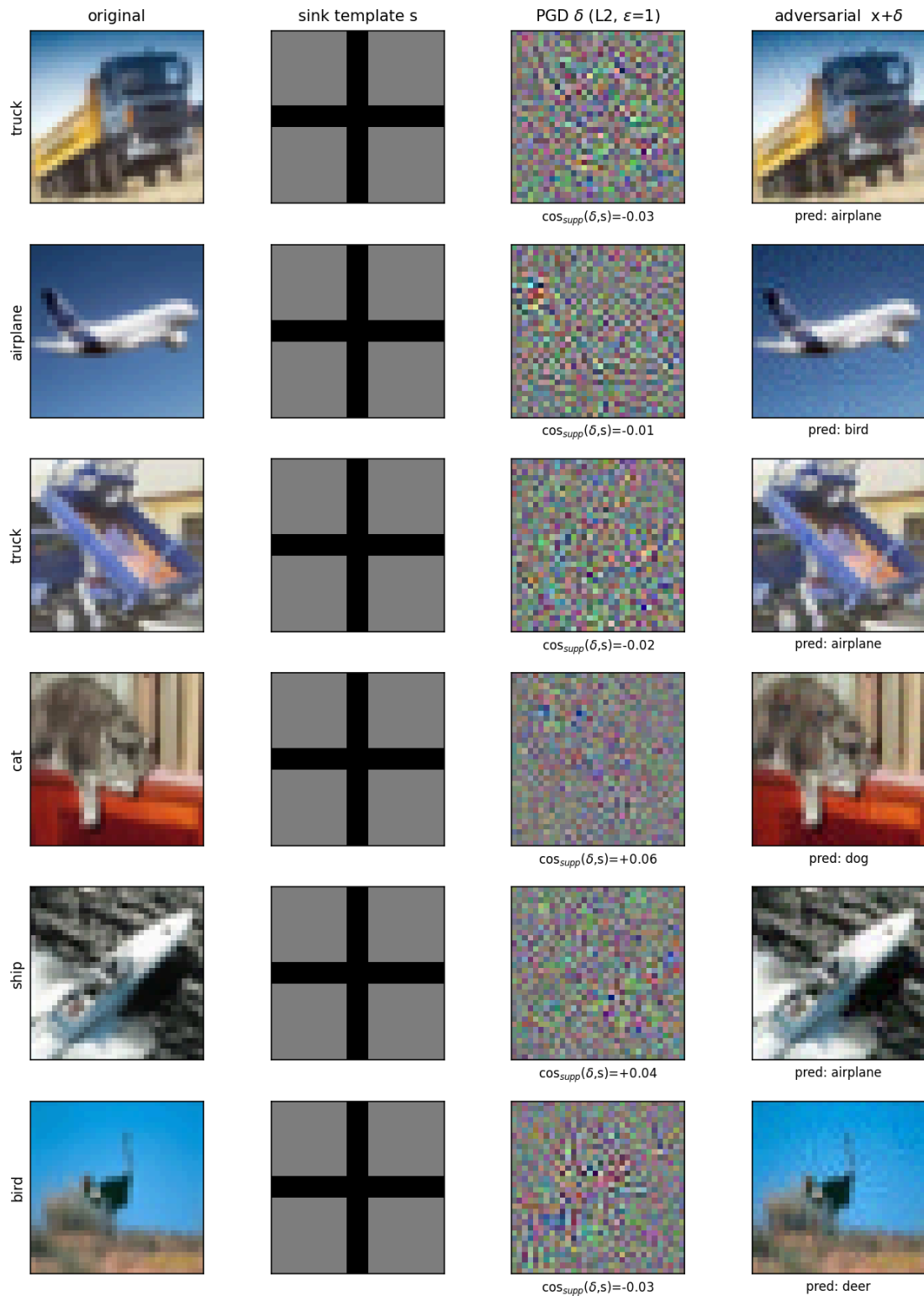


Figure 8: **What PGD draws instead of the sink.** Columns show, for several inputs, the clean image, the intended sink template, and the actual PGD perturbation. The perturbation is structured around the salient *object* pixels – edge and texture noise – and bears no resemblance to the template ($\cos \approx 0$). The attack spends its budget where the local gradient is largest, which is on the object, never on the designer’s pattern.

FGSM behaves like PGD here: both give $\text{support_cos} \approx 0$ and mass at chance, even though PGD drives robust accuracy to zero (Table 2). The one apparent exception — L_2 FGSM placing mass_frac $0.356 > 0.234$ on the cross — is central-pixel saliency, not drawing, since support_cos remains ~ 0 .

norm	attack	ϵ	robust acc	support_cos	mass_frac
L2	FGSM	0.5	0.236	+0.000	0.356
L2	FGSM	2.0	0.146	−0.002	0.355
L2	PGD	0.5	0.012	+0.002	0.282
L2	PGD	2.0	0.000	+0.004	0.258
L_∞	FGSM	0.031	0.068	+0.006	0.236
L_∞	PGD	0.031	0.000	+0.004	0.238

Table 2: **FGSM and PGD agree: no drawing, on the converged 0.92 net.** For both attacks and both norms, $\text{support_cos} \approx 0$ and mass_frac is at the chance value (0.234 for the cross). PGD is far stronger as an attack (robust accuracy $\rightarrow 0$) but no more steerable. The elevated L_2 FGSM mass reflects central-pixel saliency overlapping the cross, not reproduction of its shape.

Second, the faithful port of the toy idea: place the sink in a *label-irrelevant* direction the classifier is genuinely blind to. We aligned the converged net’s gradient toward a dense high-frequency direction — a Nyquist per-pixel checkerboard, where natural images carry almost no energy — and, as a control, toward a random direction, sweeping α .

This is where concentration finally transfers to CIFAR — but only for the high-frequency direction, and not for free. Relative to chance ($\text{energy_frac} = 3.26 \times 10^{-4}$), the no-alignment baseline already carries $1.5 \times$ (the high-frequency band naturally holds slightly more adversarial energy), and alignment lifts this to a peak of $\sim 44 \times$ at $\alpha = 6$, with $\sim 23\text{--}28 \times$ sustained at $\alpha = 8\text{--}12$. The random direction stays flat at chance for every α : concentration needs a direction the classifier is *blind to*, not merely a non-visual one.

The cost is accuracy, and the frontier is strikingly non-monotone. As α grows the model first *collapses* ($\alpha = 2 \rightarrow 0.38$ clean, reproducibly across three seeds), then *recovers* through $\alpha = 4\text{--}12$ (peak 0.69), then collapses again ($\alpha = 32 \rightarrow 0.35$). There is no cheap high-accuracy knee: any alignment strong enough to concentrate energy knocks the model off its 0.92 basin, and training only re-stabilises around $\alpha \approx 8\text{--}12$.

CIFAR-10: energy concentration transfers only for a label-irrelevant (high-freq) direction, and not for free

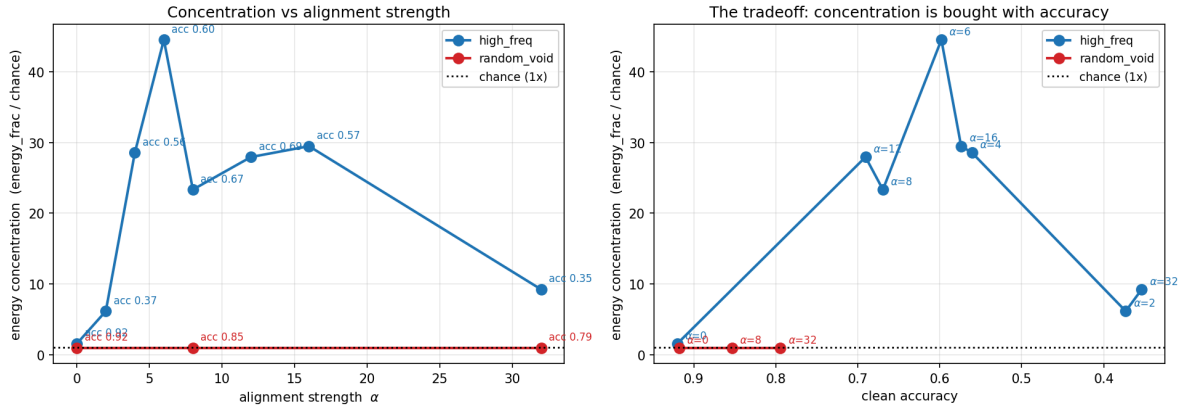


Figure 9: **Energy concentration transfers to CIFAR-10 only for a label-blind direction, and is paid for in accuracy.** Left: energy concentration (energy_frac over chance) versus alignment strength α . The high-frequency sink (blue) rises from $1.5 \times$ at $\alpha = 0$ to a peak of $44 \times$ at $\alpha = 6$, holding $23\text{--}28 \times$ at $\alpha = 8\text{--}12$; the random direction (red) never leaves the chance line. Right: the same points against clean accuracy, exposing the trade-off and its non-monotonicity — accuracy dips at $\alpha = 2$ (0.38), recovers to 0.69 near $\alpha = 12$, then collapses at $\alpha = 32$ (0.35). The usable operating region is $\alpha \approx 8\text{--}12$: $\sim 23\text{--}28 \times$ chance at 0.67–0.69 accuracy.

The mechanism is also stable across attack budget on CIFAR (Stage-3 question Q4): the concentration metrics hold as ϵ is varied, so the effect is not a single-budget artifact (Figure 10).

CIFAR-10 sink alignment vs attack budget: aligned fine-tune does not beat the converged baseline at healthy accuracy

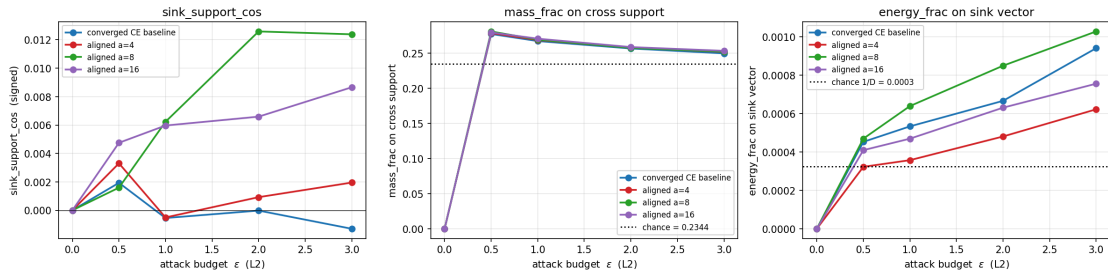


Figure 10: **Stability of the metrics versus attack budget ϵ .** support_cos, mass_frac and energy_frac as functions of the L_2 budget, for the baseline and the aligned net. The curves are smooth and monotone with no threshold effects: the (small) signal present on CIFAR is consistent across budgets rather than appearing only at one carefully chosen ϵ .

5. Synthesis and verdict

The threads pull together into a three-part story.

1. **A recognizable visual sink cannot be drawn on CIFAR.** Across five mechanisms and six patterns, on a fully-converged 0.92 network at up to $2 \times$ width, no pattern is reproduced (support_cos ≈ 0 , mass at chance). This is a *characterized* impossibility, with the capacity and dimensionality confounds explicitly ruled out.
2. **Energy concentration does transfer to CIFAR** — but only for a label-blind (high-frequency) direction, at $23\text{--}28 \times$ chance, and not for free: it costs accuracy ($0.92 \rightarrow \sim 0.68$) along a non-monotone frontier. A random direction achieves nothing.
3. **The toy proves the clean limit.** When truly unused dimensions exist, concentration reaches $36\text{--}187 \times$ chance at near-free accuracy and is robust across budget — the idealised version of result (2), with the same boundary: the sign is never controlled.

One sentence explains all of it: *a network’s input gradient can encode the class or point at a fixed direction, but not both*. Detection-grade concentration onto a label-blind subspace is real and controllable, at an accuracy price; a visible, signed drawing is blocked by exactly that tension.

A note on scope: the specification named CIFAR-100 for Stage 3. We deliberately stayed on CIFAR-10 plus the toy — once the effect fails to yield a visual drawing on the easier dataset, a harder one cannot rescue it, and the toy isolates the mechanism far more cleanly. A CIFAR-100 confirmation remains a low-risk, deferred item.

6. Conclusions and future directions

We set out to make a model betray its attacker by forcing white-box gradient attacks to draw a fixed visual symbol. The honest outcome: the strong version of this goal is unreachable for principled reasons, while a useful weaker version — forcing the attack into a known, label-blind subspace — is achievable, and was demonstrated on both a toy and CIFAR-10. The contribution is the sharp boundary between the two.

Several concrete directions follow naturally.

1. **A detector instead of a drawing.** The achievable effect is exactly what a detector needs: project an input’s perturbation onto the known basis and flag anomalous energy. This sidesteps the gradient’s dual role entirely and is the most promising next step.
2. **Off-manifold gradient sculpting.** Accuracy only constrains the model on the data manifold, yet the attack travels mostly off it. A loss that pins the function on-manifold (a KL term to a frozen classifier) while bending the gradient only at the off-manifold points the attack visits is the first formulation in which alignment and accuracy need not fight over the same gradient. The open question is the curvature cost.
3. **Designated-UAP training and architectural bottlenecks.** Train the sink as a fixed, rate-controlled universal loss-increasing direction [6] (milder than a forced label), or build a low-rank input bottleneck whose attackable subspace contains the sink by construction.
4. **Robust-feature framing.** That only a label-blind direction concentrates energy connects directly to the robust/non-robust feature view of adversarial examples [7]; characterising which subspaces are “free” for a dataset would predict the achievable concentration in advance.

Bibliography

- [1] Christian Szegedy *et al.*, “Intriguing Properties of Neural Networks,” in *International Conference on Learning Representations (ICLR)*, 2014.
- [2] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy, “Explaining and Harnessing Adversarial Examples,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [3] Aleksander Mądry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu, “Towards Deep Learning Models Resistant to Adversarial Attacks,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [4] Jonas Rauber, Wieland Brendel, and Matthias Bethge, “Foolbox: A Python Toolbox to Benchmark the Robustness of Machine Learning Models,” 2017.
- [5] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg, “BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain,” 2017.
- [6] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, and Pascal Frossard, “Universal Adversarial Perturbations,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [7] Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, and Aleksander Mądry, “Adversarial Examples Are Not Bugs, They Are Features,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [8] Alex Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” University of Toronto, 2009.